



Universidad  
Nacional  
de Loja

**UNIVERSIDAD NACIONAL DE LOJA**

**FACULTAD DE ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES**

**NO RENOVABLES**

**CARRERA DE COMPUTACIÓN**

**MATEMÁTICAS DISCRETAS**

**TÍTULO DE LA PRÁCTICA:**

**OPERACIONES BOOLEANAS, SIMPLIFICACIÓN DE EXPRESIONES, MAPAS DE  
KARNAUGH Y PROPIEDADES DEL ÁLGEBRA BOOLEANA**

**ESTUDIANTES:**

David Luna

Andy Ordoñez

Miguel Rojas

Karen Lalangui



## Reporte Técnico de Actividades

### Práctico-Experimentales Nro. 3

#### 1. Datos de Identificación del Estudiante y la Práctica

|                                              |                                                                                                                                                 |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Nombre del estudiante(s)</b>              | Miguel Rojas, David Luna, Andy Ordoñez y Karen Lalangui.                                                                                        |
| <b>Asignatura</b>                            | Matemáticas discretas                                                                                                                           |
| <b>Ciclo</b>                                 | 1er ciclo                                                                                                                                       |
| <b>Unidad</b>                                | Unidad 2: Algebra de Boole                                                                                                                      |
| <b>Resultado de aprendizaje de la unidad</b> | Utiliza eficientemente los principios de la lógica matemática, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| <b>Práctica Nro.</b>                         | 002. Fase 5 y 6                                                                                                                                 |
| <b>Título de la Práctica</b>                 | Operaciones Booleanas, Simplificación de Expresiones, Mapas de Karnaugh y Propiedades del Álgebra Booleana                                      |
| <b>Nombre del Docente</b>                    | Mario Enrique Cueva Hurtado                                                                                                                     |
| <b>Fecha</b>                                 | Semana 9 y 10: del 1 al 12 de junio                                                                                                             |
| <b>Horario</b>                               | Lunes, Martes, jueves 07:30 – 09:30                                                                                                             |



---

|                                        |                                                                                      |
|----------------------------------------|--------------------------------------------------------------------------------------|
| <b>Lugar</b>                           | Aula de la Carrera de Computación / Laboratorio de Computación / Aula Virtual (Zoom) |
| <b>Tiempo planificado en el Sílabo</b> | 18 horas (3 horas por sesión APE)                                                    |

## 2. Objetivo(s) de la Práctica

- Comprender los principios fundamentales del álgebra de Boole y su relación con la lógica proposicional estudiada en la Unidad 1.
- Aplicar las operaciones booleanas básicas (AND, OR, NOT, XOR, XNOR, NAND, NOR) en la construcción y simplificación de expresiones booleanas.
- Dominar las técnicas de simplificación de expresiones booleanas: algebraica, por mapas de Karnaugh (2, 3 y 4 variables), y propiedades del álgebra booleana.
- Diseñar funciones lógicas a partir de especificaciones dadas, expresándolas en diferentes formas (suma de productos, producto de sumas) y minimizándolas.
- Relacionar el álgebra de Boole con aplicaciones prácticas en computación: circuitos lógicos, diseño digital, consultas SQL y programación.

## 3. MATERIALES Y REACTIVOS.

- Pizarra y marcadores de colores
- Proyector multimedia
- Hojas de trabajo con ejercicios de operaciones booleanas y mapas de Karnaugh
- Plantillas impresas de mapas de Karnaugh (2, 3 y 4 variables)



- Guía de simplificación algebraica de expresiones booleanas
- Calculadora científica.

#### **4. EQUIPOS Y HERRAMIENTAS**

- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entregas y foros
- Laboratorio de computación para prácticas
- Herramientas colaborativas: Google Drive, Wiki
- Simuladores de circuitos lógicos online (Logisim, CircuitVerse)
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

#### **5. Procedimiento / Metodología Ejecutada**

- ✓ Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital y programación.
- ✓ Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales): los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta.
- ✓ Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.



- 
- ✓ Práctica integradora: cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.
  - ✓ Los equipos entregan el portafolio integrador de la unidad que incluya: todos los talleres de simplificación booleana, mapas de Karnaugh resueltos, diseño de funciones lógicas, y el informe reflexivo sobre optimización.
  - ✓ Sesión de socialización: los equipos presentan el caso aplicado que diseñaron (sumador, comparador, etc.), explicando el proceso de diseño, las simplificaciones realizadas y las ventajas obtenidas.
  - ✓ Coevaluación entre equipos mediante rúbricas. Retroalimentación del docente sobre el portafolio y la presentación. Análisis de casos aplicados a la computación.

## **6. Resultados**

- ✓ **Karen Lalangui**

## REPORTE TÉCNICO DE ACTIVIDADES PRÁCTICO- EXPERIMENTALES NRO. 2 FASE 5 y 6.

### RESULTADO.

Paso 2: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital y programación.

#### 1. LEY DE CONSENSO

Aplicación: En circuitos lógicos permite reducir la cantidad de compuertas utilizadas disminuyendo:

- Costo del circuito
- Consumo de energía.

Ejemplo: En un procesador digital simplificar una función lógica permite crear más rápidos y eficientes.

Ejercicio:

$$\begin{aligned} & \rightarrow AB + A'C + BC + ABC \\ & AB + A'C + ABC \rightarrow \text{Consenso} \\ & AB(1+C) + A'C \rightarrow \text{Distributiva} \\ & AB + A'C \rightarrow \text{Rta.} \end{aligned}$$

#### 2. LEY EXPANSIÓN SHANNON

Aplicación: Se utiliza en:

- diseño de multiplexores
- Creación de memorias y capacitores.

Ejemplo: Un multiplexor selecciona una entrada u otra dependiendo del valor de una señal de control aplicando el principio de Shannon.

Ejercicio:

$$\rightarrow F(A, B, C) = AB + C$$

Expandir respecto a "B".

1. Cuando  $B=1$ :  $F(A, 1, C) = A(1) + C = A + C$
2. Cuando  $C=0$ :  $F(A, 0, C) = A(0) + C = C$ .
3. Resultado:  $B(A+C) + B'C$

### 3. RESOLUCIÓN BOOLEANA.

Aplicación = Métodos automáticos de diseño de hardware, y optimizar sistemas digitales.

Ejemplo: Sistema de alarma.

A = puerta activa

$\bar{A}$  = puerta no activada

B = Sensor de movimiento.

C = alarma encendida.

$$(A+B)(\bar{A}+C) =$$

$$A\bar{A} + AC + \bar{A}B + BC \rightarrow \text{distributiva}$$

$$0 + AC + \bar{A}B + BC \rightarrow \text{negación}$$

$$AC + \bar{A}B \rightarrow \text{Consenso}$$

$$(A+\bar{A})(\bar{A}+C)(B+C) \rightarrow \text{distributiva}$$

$$B+C \rightarrow \text{Red.}$$

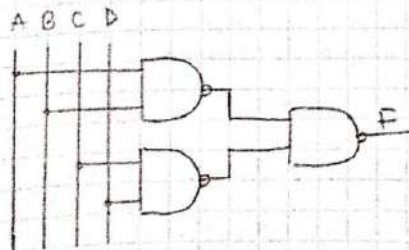
Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (compuertas universales). Los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta.

#### 1. EJERCICIO 1.

$$F = AB + CD$$

$$F = \overline{(\overline{AB + CD})} \rightarrow \text{Doble negación}$$

$$F = \overline{(\bar{A}\bar{B})(\bar{C}\bar{D})} \rightarrow \text{Morgan.}$$



### EJERCICIO 2.

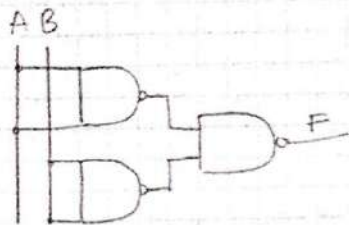
$$F = A + B$$

$$F = \overline{\overline{A + B}} \quad \text{Doble negación}$$

$$F = \overline{(\overline{A} \overline{B})} \quad \text{Morgan}$$

$$\begin{aligned} \overline{A} &= \overline{A A} \\ \overline{B} &= \overline{B B} \end{aligned} \quad \left. \vphantom{\begin{aligned} \overline{A} &= \overline{A A} \\ \overline{B} &= \overline{B B} \end{aligned}} \right\} \text{Inversores NAND}$$

$$F = \overline{(\overline{A A}) (\overline{B B})} \rightarrow \text{Plata}$$



### EJERCICIO 3.

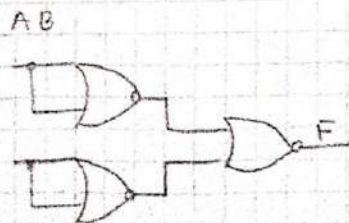
$$F = AB$$

$$F = \overline{\overline{AB}} \quad \text{Doble negación}$$

$$F = \overline{(\overline{A + B})} \rightarrow \text{Morgan}$$

$$\begin{aligned} \overline{A} &= \overline{A + A} \\ \overline{B} &= \overline{B + B} \end{aligned} \quad \left. \vphantom{\begin{aligned} \overline{A} &= \overline{A + A} \\ \overline{B} &= \overline{B + B} \end{aligned}} \right\} \text{Inversores NOR}$$

$$F = \overline{(\overline{A + A}) + (\overline{B + B})} \rightarrow \text{Plata}$$





---

## Informe Reflexivo: El Impacto de la Optimización Lógica en la Sostenibilidad y Viabilidad de los Sistemas Computacionales

### 1. Introducción

En el ámbito de la ingeniería de la computación, el diseño de sistemas digitales eficientes no se limita exclusivamente a maximizar la velocidad de procesamiento o la capacidad de almacenamiento. Existe una dimensión física y operativa que vincula directamente la abstracción matemática con los recursos materiales del planeta. La optimización lógica, fundamentada en herramientas como el álgebra de Boole y los diagramas de simplificación como los mapas de Karnaugh, permite reducir expresiones complejas de términos lógicos a sus formas mínimas equivalentes.

El presente informe analiza cómo este proceso de simplificación teórica trasciende el papel y el software de simulación, impactando de manera directa en la reducción de costos de manufactura, la optimización del consumo energético y la mitigación de la huella ambiental durante la fabricación y operación de circuitos integrados.

### 2. Reducción de Costos en la Fabricación de Circuitos

El impacto económico de la simplificación booleana en la industria de los semiconductores es directo y escalable. Cada término o variable en una función lógica no optimizada se traduce físicamente en compuertas lógicas adicionales y, en consecuencia, en un mayor número de transistores interconectados dentro de un sustrato de silicio.

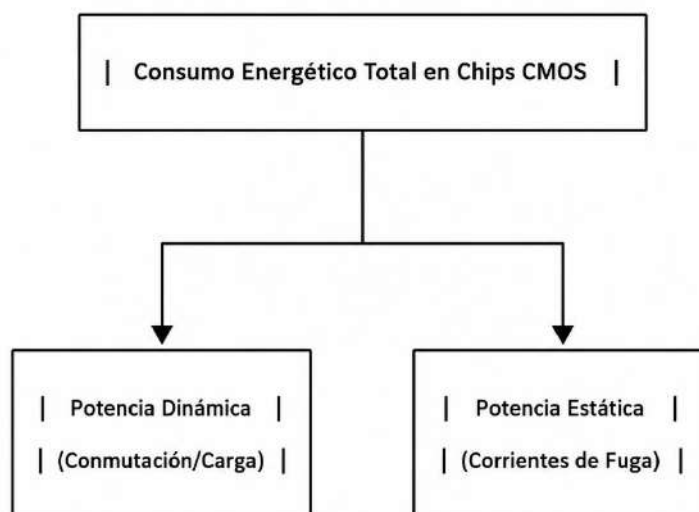
Al aplicar algoritmos de optimización se logran los siguientes beneficios económicos:

- **Optimización del área del chip (*Die Size*):** Al reducir la cantidad de transistores necesarios para ejecutar una función funcionalmente idéntica, el área física que ocupa el circuito integrado en la oblea de silicio disminuye de forma notable.

- **Maximización del rendimiento de la oblea (Yield Rate):** La fabricación de microchips se realiza en obleas circulares de silicio. Chips más pequeños permiten proyectar una mayor cantidad de circuitos por oblea. Asimismo, dado que los defectos físicos de fabricación se distribuyen de manera aleatoria, un área menor por circuito reduce estadísticamente la probabilidad de que un chip resulte defectuoso, aumentando la productividad y reduciendo el costo unitario de producción.
- **Reducción de costos de interconexión y encapsulado:** Circuitos más simples requieren menos capas de metalización para el enrutamiento de señales y encapsulados menos complejos, abaratando los costos logísticos y de ensamblaje final.

### 3. Disminución del Consumo Energético en Sistemas Computacionales

La eficiencia energética en el hardware computacional moderno está gobernada por leyes físicas que se ven directamente afectadas por la complejidad del circuito lógico. El consumo de energía en los transistores de tecnología complementaria de óxido metálico semiconductor (CMOS) se divide en dos componentes críticos:



- **Potencia Dinámica:** Ocurre principalmente durante la conmutación de los estados lógicos (transiciones de 0 a 1 y viceversa). Cada compuerta lógica posee una capacitancia parásita intrínseca que debe cargarse y descargarse en cada ciclo de reloj. La eliminación de compuertas redundantes mediante la simplificación booleana disminuye la capacitancia total del circuito, lo que reduce directamente la energía requerida para su funcionamiento dinámico.
- **Potencia Estática (Corrientes de Fuga / *Leakage*):** En las tecnologías de manufactura a nanoescala actuales, los transistores experimentan fugas de corriente eléctrica de manera constante, incluso cuando se encuentran en estado de reposo o inactividad. La optimización lógica reduce el número total de transistores en el silicio, acotando de forma proporcional las pérdidas energéticas pasivas por fugas de corriente.
- **Carga Térmica y Enfriamiento Indirecto:** Un menor consumo energético interno se traduce en una menor disipación de calor (efecto Joule). Al reducir la temperatura de operación de los circuitos, se disminuyen los requerimientos de sistemas externos de disipación térmica (como ventiladores o sistemas de refrigeración líquida), generando un ahorro energético secundario a nivel de sistema.

#### 4. Mitigación de la Huella Ambiental y Sostenibilidad

La ingeniería verde aplicada a la computación encuentra en la optimización lógica una de sus herramientas más limpias, ya que previene el impacto ambiental desde la fase de diseño de la arquitectura (*sostenibilidad por diseño*).



|                                        |                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dimensión Ambiental</b>             | <b>Impacto de la Optimización Lógica</b>                                                                                                                                                                                                                                                                                      |
| <b>Extracción de Recursos</b>          | La reducción en el tamaño de los chips disminuye la demanda de silicio de alta pureza, metales conductores y tierras raras. Esto reduce la actividad minera y el impacto ambiental asociado a la extracción y procesamiento de estos materiales.                                                                              |
| <b>Huella de Carbono Operativa</b>     | La optimización de los circuitos reduce el consumo de energía en dispositivos electrónicos y centros de datos. Como consecuencia, disminuye la demanda de electricidad y las emisiones de gases de efecto invernadero derivadas de la generación de energía.                                                                  |
| <b>Residuos Electrónicos (E-waste)</b> | Los circuitos optimizados generan menos calor y sufren un menor desgaste por estrés térmico. Esto prolonga la vida útil del hardware, retrasa su reemplazo y reduce la cantidad de residuos electrónicos generados.                                                                                                           |
| <b>Residuos Electrónicos (E-waste)</b> | Los circuitos integrados optimizados que operan a menores temperaturas sufren un desgaste físico menor derivado del estrés térmico. Esto prolonga el ciclo de vida útil de los componentes lógicos, retrasando la obsolescencia del hardware y reduciendo el volumen de desechos electrónicos que contaminan los ecosistemas. |



## 5. Conclusión

El análisis efectuado permite concluir que la optimización lógica constituye un puente indispensable entre las matemáticas discretas y el desarrollo tecnológico sostenible. La simplificación de expresiones booleanas y la reducción de compuertas lógicas no deben considerarse únicamente ejercicios de abstracción algorítmica para mejorar el rendimiento de cómputo. Su aplicación sistemática representa una estrategia de ingeniería crítica que impacta favorablemente en la economía de los procesos industriales, reduce la demanda energética de la infraestructura digital y promueve una manufactura tecnológica más responsable con el medio ambiente. El diseño eficiente es, en consecuencia, el primer paso hacia una computación ecológica y sostenible.

Andrés Vladimir Ochoa Nunez

## Comparador Digital de 1 bit

Un comparador digital de 1 bit toma dos variables de entrada (A y B) y determina su relación de magnitud. Para esto necesitamos tres salidas que indiquen si A es mayor, igual o menor que B.

### 1. Construcción de Tabla de Verdad

Primero definimos todas las combinaciones posibles para nuestras dos entradas de 1 bit (A y B) y evaluamos las tres salidas deseadas:

- Mayor:  $A > B$
- Igual:  $A = B$
- Menor:  $A < B$

| Entrada A | Entrada B | Mayor ( $A > B$ ) | Igual ( $A = B$ ) | Menor ( $A < B$ ) |
|-----------|-----------|-------------------|-------------------|-------------------|
| 0         | 0         | 0                 | 1                 | 0                 |
| 0         | 1         | 0                 | 0                 | 1                 |
| 1         | 0         | 1                 | 0                 | 0                 |
| 1         | 1         | 0                 | 1                 | 0                 |

### 2. Mapas de Karnaugh

#### Mapa para Mayor ( $A > B$ ):

Solo hay un "1" lógico en la celda donde  $A=1$  y  $B=0$ . Como está aislado, no se puede agrupar con otras celdas.

#### Mapa para Menor ( $A < B$ ):

Solo hay un "1" lógico en la celda donde  $A=0$  y  $B=1$ . Tampoco hay posibilidad de formar lazos.

#### Mapa para Igual ( $A = B$ ):

Hay "1s" lógicos en las celdas (0,0) y (1,1). Estos unos están en diagonal, lo que significa que en un mapa de Karnaugh estándar no se pueden agrupar.

El hecho de que el mapa de Karnaugh no permita agrupaciones indica la expresión en suma de productos ya está en su forma más reducida usando compuertas básicas (AND/OR/NOT), pero es el punto de partida perfecto para la simplificación por álgebra de Boole usando compuertas especiales.

### 3. Simplificación mediante Álgebra Booleana

A partir de los "1s" de la tabla de verdad y los mapas, extraemos las ecuaciones en Suma de Productos (SDP) y aplicamos las propiedades del álgebra de Boole.

Para la salida Mayor ( $A > B$ ):

La función se activa solo cuando A es verdadera y B es falsa.

$$\text{Mayor} = A \cdot \bar{B}$$

Para la salida Menor ( $A < B$ ):

La función se activa solo cuando A es falsa y B es verdadera.

$$\text{Menor} = \bar{A} \cdot B$$

Para la salida Igual ( $A = B$ ):

Extraemos la ecuación inicial sumando los minterminos donde la salida es 1:

$$\text{Igual} = (A \cdot \bar{B}) + (\bar{A} \cdot B)$$

Aquí aplicamos el álgebra booleana. Esta estructura lógica corresponde exactamente a la definición de la compuerta XNOR (o la equivalencia lógica). Podemos reescribir esta expresión de manera más elegante y optimizada para su implementación física:

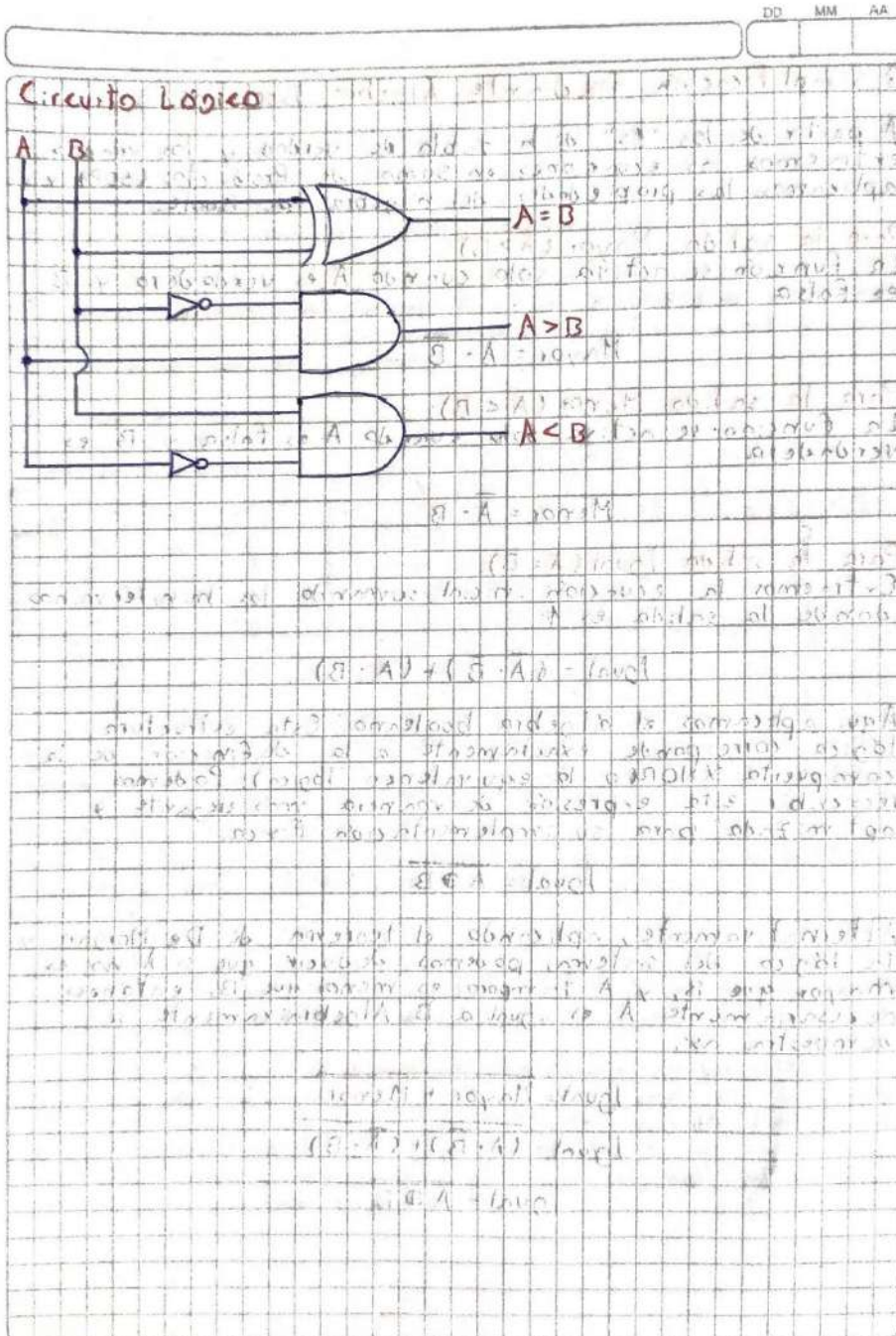
$$\text{Igual} = A \oplus B$$

Alternativamente, aplicando el teorema de De Morgan y la lógica del sistema, podemos deducir que si A no es mayor que B, y A tampoco es menor que B, entonces necesariamente A es igual a B. Algebraicamente se demuestra así:

$$\text{Igual} = \text{Mayor} + \text{Menor}$$

$$\text{Igual} = (A \cdot \bar{B}) + (\bar{A} \cdot B)$$

$$\text{Igual} = A \oplus B$$



## 7. Preguntas de Control

1. Simplifique la siguiente expresión booleana utilizando el método de Karnaugh (4 variables):

$F(A,B,C,D) = \Sigma m(0,1,2,5,6,7,8,9,10,14)$ . Muestre el mapa de Karnaugh, los grupos formados y la expresión simplificada resultante. Compare el resultado con la simplificación algebraica.

**Función a simplificar:**  $F(A,B,C,D) = \Sigma m(0,1,2,5,6,7,8,9,10,14)$

### Paso 1: Tabla de Verdad Completa

| m# | A | B | C | D | F |
|----|---|---|---|---|---|
| 0  | 0 | 0 | 0 | 0 | 1 |
| 1  | 0 | 0 | 0 | 1 | 1 |
| 2  | 0 | 0 | 1 | 0 | 1 |
| 3  | 0 | 0 | 1 | 1 | 0 |
| 4  | 0 | 1 | 0 | 0 | 0 |
| 5  | 0 | 1 | 0 | 1 | 1 |
| 6  | 0 | 1 | 1 | 0 | 1 |
| 7  | 0 | 1 | 1 | 1 | 1 |
| 8  | 1 | 0 | 0 | 0 | 1 |

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| 9  | 1 | 0 | 0 | 1 | 1 |
| 10 | 1 | 0 | 1 | 0 | 1 |
| 11 | 1 | 0 | 1 | 1 | 0 |
| 12 | 1 | 1 | 0 | 0 | 0 |
| 13 | 1 | 1 | 0 | 1 | 0 |
| 14 | 1 | 1 | 1 | 0 | 1 |
| 15 | 1 | 1 | 1 | 1 | 0 |

**Minterms donde F = 1:** 0, 1, 2, 5, 6, 7, 8, 9, 10, 14

**Minterms donde F = 0:** 3, 4, 11, 12, 13, 15

## Paso 2: Construcción del Mapa de Karnaugh

| AB \ CD | 00      | 01      | 11      | 10      |
|---------|---------|---------|---------|---------|
| 00      | 1 (m0)  | 1 (m1)  | 0 (m3)  | 1 (m2)  |
| 01      | 0 (m4)  | 1 (m5)  | 1 (m7)  | 1 (m6)  |
| 11      | 0 (m12) | 0 (m13) | 0 (m15) | 1 (m14) |
| 10      | 1 (m8)  | 1 (m9)  | 0 (m11) | 1 (m10) |

Grupo 1:

{0,1,8,9}

Grupo 2:

{0,2,8,10}

Grupo 3: {5,7}

Grupo 4: {6,14}

### Paso 3: Expresión Simplificada por Karnaugh

$$F(A,B,C,D) = B'C' + B'D' + A'BD + BCD'$$

### Paso 4: Comparación con Simplificación Algebraica

$$\begin{aligned} F &= A'B'C'D' + A'B'C'D + A'B'CD' + A'BC'D + A'BCD' + A'BCD \\ &+ AB'C'D' + AB'C'D + AB'CD' + ABCD' \end{aligned}$$

#### Paso 1 — Grupo A $\rightarrow m0 + m1$

$$\begin{aligned} &A'B'C'D' + A'B'C'D \\ = &A'B'C'(D' + D) \leftarrow \text{Factor común: } A'B'C' \\ = &A'B'C'(1) \leftarrow \text{Complemento: } D + D' = 1 \\ = &A'B'C' \leftarrow \text{Identidad: } x \cdot 1 = x \end{aligned}$$

#### Paso 2 — Grupo B $\rightarrow m0 + m2$

$$A'B'C'D' + A'BCD'$$



$$= A'B'D'(C' + C) \leftarrow \text{Factor común: } A'B'D'$$

$$= A'B'D'(1) \leftarrow \text{Complemento: } C + C' = 1$$

$$= A'B'D' \leftarrow \text{Identidad: } x \cdot 1 = x$$

### Paso 3 — Grupo C $\rightarrow$ m8 + m9

$$AB'C'D' + AB'C'D$$

$$= AB'C'(D' + D) \leftarrow \text{Factor común: } AB'C'$$

$$= AB'C'(1) \leftarrow \text{Complemento: } D + D' = 1$$

$$= AB'C' \leftarrow \text{Identidad: } x \cdot 1 = x$$

### Paso 4 — Grupo D $\rightarrow$ m8 + m10

$$AB'C'D' + AB'CD'$$

$$= AB'D'(C' + C) \leftarrow \text{Factor común: } AB'D'$$

$$= AB'D'(1) \leftarrow \text{Complemento: } C + C' = 1$$

$$= AB'D' \leftarrow \text{Identidad: } x \cdot 1 = x$$

### Paso 5 — Fusionar Grupos A y C $\rightarrow$ A'B'C' + AB'C'

$$A'B'C' + AB'C'$$

$$= B'C'(A' + A) \leftarrow \text{Factor común: } B'C'$$

$$= B'C'(1) \leftarrow \text{Complemento: } A + A' = 1$$

$$= B'C' \leftarrow \text{Identidad: } x \cdot 1 = x \quad \checkmark$$

---

**Paso 6 — Fusionar Grupos B y D  $\rightarrow A'B'D' + AB'D'$**

$$\begin{aligned} & A'B'D' + AB'D' \\ = & B'D'(A' + A) \leftarrow \text{Factor común: } B'D' \\ = & B'D'(1) \leftarrow \text{Complemento: } A + A' = 1 \\ = & B'D' \leftarrow \text{Identidad: } x \cdot 1 = x \quad \checkmark \end{aligned}$$

**Paso 7 — Grupo E  $\rightarrow m5 + m7$**

$$\begin{aligned} & A'BC'D + A'BCD \\ = & A'BD(C' + C) \leftarrow \text{Factor común: } A'BD \\ = & A'BD(1) \leftarrow \text{Complemento: } C + C' = 1 \\ = & A'BD \leftarrow \text{Identidad: } x \cdot 1 = x \quad \checkmark \end{aligned}$$

**Paso 8 — Grupo F  $\rightarrow m6 + m14$**

$$\begin{aligned} & A'BCD' + ABCD' \\ = & BCD'(A' + A) \leftarrow \text{Factor común: } BCD' \\ = & BCD'(1) \leftarrow \text{Complemento: } A + A' = 1 \\ = & BCD' \leftarrow \text{Identidad: } x \cdot 1 = x \quad \checkmark \end{aligned}$$

**Paso 9 — Reunir todos los términos simplificados**

$$F = B'C' + B'D' + A'BD + BCD'$$



2. Explique por qué las compuertas NAND y NOR se consideran "universales". Demuestre cómo implementar las operaciones AND, OR y NOT utilizando únicamente compuertas NAND. ¿Qué implicaciones tiene esto en el diseño de circuitos integrados?

### **Paso 1: ¿Qué significa ser una compuerta 'universal'?**

Una compuerta lógica se denomina UNIVERSAL cuando por sí sola puede implementar CUALQUIER función lógica booleana, incluyendo las operaciones básicas NOT, AND y OR. Esto se basa en el Teorema de Completitud Funcional del álgebra de Boole.

Para demostrar que una compuerta es universal, basta con mostrar que puede implementar las tres operaciones fundamentales:

1. **NOT (inversor)** — operación complemento
2. **AND** — producto lógico
3. **OR** — suma lógica

### **2. La Compuerta NAND: Definición y Tabla de Verdad**

La compuerta **NAND** (Not **AND**) realiza la operación AND seguida de una inversión. Su ecuación característica es:

$$F = \text{NAND}(A, B) = (A \cdot B)'$$

Su tabla de verdad muestra que la salida es 0 únicamente cuando AMBAS entradas son 1:

| A | B | $A \cdot B$ | NAND(A,B) |
|---|---|-------------|-----------|
| 0 | 0 | 0           | 1         |
| 0 | 1 | 0           | 1         |
| 1 | 0 | 0           | 1         |
| 1 | 1 | 1           | 0         |

### 3. Demostración: NOT, AND y OR con Compuertas NAND

#### 3.1 Operación NOT (Inversor)

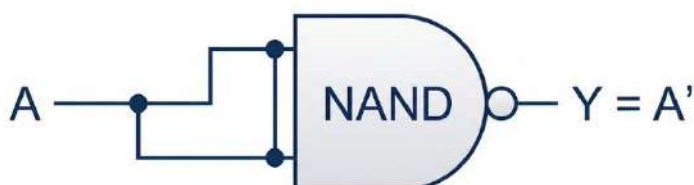
Al conectar ambas entradas de una compuerta NAND al mismo valor A, se obtiene un inversor:

**Demostración algebraica:**

$$\begin{aligned}
 F &= \text{NAND}(A, A) = (A \cdot A)' \\
 &= (A)' \quad [\text{idempotencia: } A \cdot A = A]
 \end{aligned}$$

$$F = A' \quad \checkmark$$

**Diagrama:**



### 3.2 Operación AND

Una compuerta AND es la inversa de NAND. Para obtenerla se aplica la **ley de doble negación**: negar dos veces un valor lo restaura.

**Demostración algebraica:**

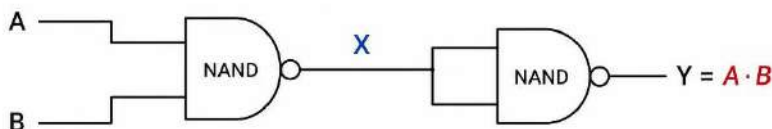
$$X = \text{NAND}(A, B) = (A \cdot B)'$$

$$F = \text{NAND}(X, X) = (X \cdot X)' = X'$$

$$= ((A \cdot B)')'$$

$$F = A \cdot B \quad \checkmark \quad [\text{doble negación}]$$

**Diagrama (2 compuertas NAND):**



### 3.3 Operación OR

Para obtener OR se aplica el **Teorema de De Morgan**:  $A + B = (A' \cdot B')'$ . Se invierten las entradas con dos compuertas NAND-NOT y se aplica una NAND final.

**Demostración algebraica:**

$$X = \text{NAND}(A, A) = A'$$

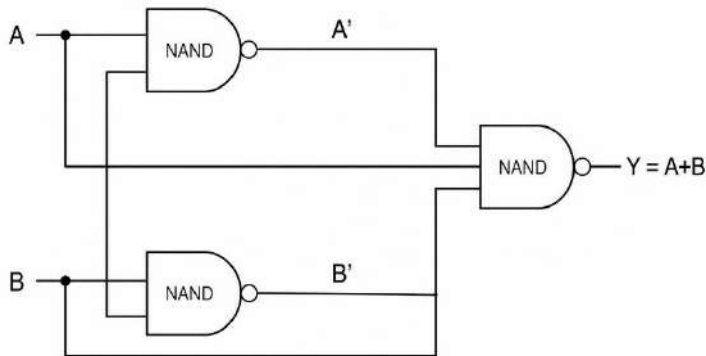
$$Y = \text{NAND}(B, B) = B'$$

$$F = \text{NAND}(X, Y) = (A' \cdot B')'$$

$$= A'' + B'' \quad [\text{De Morgan}]$$

$$F = A + B \quad \checkmark$$

**Diagrama (3 compuertas NAND):**



#### 4. Implicaciones en el Diseño de Circuitos Integrados

La universalidad de las compuertas NAND y NOR no es solo una propiedad teórica; tiene consecuencias **directas y profundas** en la fabricación de microprocesadores, FPGAs y cualquier circuito integrado digital moderno.



#### 4.1 Reducción de costos de manufactura

Fabricar un chip de silicio que repite miles de millones de veces **la misma estructura geométrica** (una compuerta NAND) es significativamente más barato que diseñar máscaras de fabricación distintas para AND, OR, NOT, etc. La estandarización de una sola celda lógica reduce el tiempo de diseño y el costo de las fotomáscaras utilizadas en la litografía.

#### 4.2 Optimización del área en silicio (Tecnología CMOS)

En la tecnología **CMOS** —base de todos los chips modernos—, una compuerta NAND requiere únicamente **4 transistores**, mientras que una compuerta AND equivalente necesita **6 transistores** (NAND + inversor). Construir circuitos directamente con NAND elimina los transistores redundantes, **reduciendo el área del chip** y permitiendo mayor densidad de integración.

#### 4.3 Menor consumo de energía y mayor velocidad

Al usar menos transistores por celda lógica fundamental se reduce la **capacitancia parásita**. Esto se traduce en dos beneficios directos: menor **retardo de propagación** (las señales se propagan más rápido) y menor **disipación de calor**, un factor crítico en diseños de alta densidad como los procesadores modernos.

---

#### 4.4 Estandarización e inventario simplificado

Históricamente, con la serie de chips **TTL 7400**, los ingenieros podían resolver cualquier problema de diseño adquiriendo un único tipo de componente (p. ej., el **74HC00**: cuatro compuertas NAND en un solo encapsulado DIP). Esto simplificó enormemente el inventario de componentes en los laboratorios y la producción industrial.

#### 4.5 Base del diseño con FPGAs y PLDs

Las **FPGAs** (Field-Programmable Gate Arrays) y los dispositivos lógicos programables (PLD) organizan su lógica interna en bloques basados en LUTs (Look-Up Tables) que funcionalmente equivalen a redes de compuertas NAND. Esto permite **programar cualquier función lógica** dentro del mismo dispositivo de silicio, sin alterar el hardware físico.

### 8. Conclusiones

✓ **Karen Lalangui**

Durante el desarrollo de la práctica, se puede concluir que el estudio de las leyes de consenso, la expansión de Shannon y la resolución booleana permiten simplificar y optimizar una expresión lógica. La ley de consenso nos permite eliminar términos redundantes sin cambiar el funcionamiento de un circuito, mientras que la expansión de Shannon facilita el análisis de funciones booleanas según el valor binario que se le asigne a la variable, siendo útil en el diseño de multiplexores y sistemas digitales. Por otro lado, la resolución booleana nos permite



combinar cláusulas lógicas para eliminar variables y obtener expresiones más simples para la implementación de un circuito lógico.

✓ **Miguel Rojas**

Al evaluar la simplificación booleana y el uso de compuertas universales, queda claro que la optimización lógica va más allá de la teoría y define la eficiencia real del hardware. Reducir expresiones mediante mapas de Karnaugh minimiza físicamente los circuitos integrados, lo que se traduce en un menor uso de silicio, conexiones más cortas y una menor capacitancia. En la computación, esto no solo acelera la velocidad de los procesadores, sino que disminuye drásticamente el consumo de energía y la disipación de calor. Además, este enfoque reduce el impacto ambiental de la manufactura tecnológica al optimizar recursos, bajar costos de producción y frenar el desecho electrónico. Como futuros profesionales, dominar estas técnicas es una responsabilidad técnica y ecológica fundamental para diseñar un software y hardware verdaderamente sostenibles.

✓ **Andy Ordoñez**

En conclusión, la práctica del álgebra de Boole demuestra que la optimización de expresiones mediante mapas de Karnaugh, leyes de consenso, la expansión de Shannon y la resolución booleana trasciende la teoría para determinar la eficiencia real del hardware. Al eliminar redundancias y simplificar las funciones lógicas, se logra un impacto físico directo en los circuitos integrados: menor uso de silicio, conexiones más cortas y baja capacitancia, lo que



---

acelera los procesadores y disminuye drásticamente tanto el consumo de energía como la disipación de calor. De este modo, el uso de compuertas universales y la minimización lógica no solo optimizan el rendimiento técnico, sino que reducen los costos y la huella ambiental de la manufactura tecnológica, convirtiendo este aprendizaje en una responsabilidad técnica y ecológica fundamental para el desarrollo de sistemas computacionales verdaderamente sostenibles.

✓ **Santiago Luna**

A través del desarrollo de la práctica, se concluye que el álgebra de Boole constituye el puente matemático fundamental entre la lógica proposicional y la arquitectura computacional moderna, demostrando que la correcta aplicación y combinación de sus operaciones básicas y universales permite modelar y estructurar cualquier sistema de decisión digital. Mediante el dominio de las técnicas de simplificación analítica y visual, específicamente el uso de Mapas de Karnaugh de hasta cuatro variables, se logró transformar especificaciones funcionales en expresiones lógicas mínimas —tanto en suma de productos como en producto de sumas—, validando que la optimización algebraica reduce drásticamente la complejidad y el costo físico en el diseño de circuitos digitales. Finalmente, este proceso evidenció que el pensamiento booleano trasciende el hardware, consolidándose como una competencia transversal crítica en la computación, con aplicaciones directas que van desde la construcción de procesadores hasta



la eficiencia en el desarrollo de software, reflejada en la optimización de consultas SQL y en la lógica de control de la programación estructurada.

## 9. ENLACES PORTAFOLIO

✓ Miguel Rojas

<https://drive.google.com/drive/folders/1Hnz3cJ3ie1dluOld3tD686Mys6JLS78J>

✓ David Luna

[https://drive.google.com/drive/folders/1GvVurHh9NRmcFImL\\_fJoNVTgpvIz8p1?usp=sharing](https://drive.google.com/drive/folders/1GvVurHh9NRmcFImL_fJoNVTgpvIz8p1?usp=sharing)

✓ Andy Ordoñez

[https://drive.google.com/drive/folders/1ZCgr\\_i7qLVmC78tMXXnhozWd6zOLw3hr?usp=sharing](https://drive.google.com/drive/folders/1ZCgr_i7qLVmC78tMXXnhozWd6zOLw3hr?usp=sharing)

✓ Karen Lalangui

<https://github.com/valentinalalangui05-source/Portafolio-Matematicas-discretas/edit/main/README.md>

## 10. Anexos

✓ Miguel Rojas

### 1ª Ley de consenso

$$1. \quad A B + A' C + B C + A B C$$

$$\begin{aligned} A B + A' C + A B C &\rightarrow \text{Consenso} \\ A B (1 + C) + A' C &\rightarrow \text{Distributiva} \\ A B (1) + A' C &\rightarrow \text{Identidad} \\ A B + A' C &\rightarrow \text{Rta} \end{aligned}$$

#### Aplicación

En circuitos lógicos permite reducir las cantidades de componentes utilizados, disminuyendo

- Costo del circuito
- Consumo de energía
- Espacio físico de un chip

#### Ejemplo

En un procesador digital, simplificar una función lógica permite crear circuitos más rápidos y eficientes.

### 2ª Expansión de Shannon

Expandir respecto a B.

$$F(A, B, C) = A B + C$$

1ª Cuando  $B=1$

$$F(A, 1, C) = A(1) + C = A + C$$

2ª Cuando  $B=0$

$$F(A, 0, C) = A(0) + C = C$$

3ª Respuesta

$$F(A, B, C) = B(A + C) + B' C$$

#### Aplicación

Se utiliza en

- Diseño de multiplexores
- Generación de memoria y capacitores

#### Ejemplo

Un multiplexor selecciona una entrada u otra dependiendo del valor de una señal de control explicando el principio de Shannon.

### 3. Reducción booleana

$$+ (A + B) (A' + C) =$$

$$\begin{aligned} A \bar{A} + A C + \bar{A} B + B C &= \rightarrow \text{Distributing} \\ 0 + A C + \bar{A} B + B C &\rightarrow \text{Negación} \\ A C + B C &\rightarrow \text{Consenso} \\ B + C & \end{aligned}$$

### Aplicación

Métodos de diseño de hardware automático  
Optimizar sistemas digitales

### Ejemplo

Sistema de alarma

$A$  = Sensor de puerta activado  
 $\bar{A}$  = Puerta no activado  
 $B$  = Sensor de movimiento  
 $C$  = Alarma encendida

Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND y NOR: los estudiantes diseñarán circuitos equivalentes usando un solo tipo de compuerta

Ejercicio 1:

$$F = A \cdot B + C \cdot D$$

Desarrollo

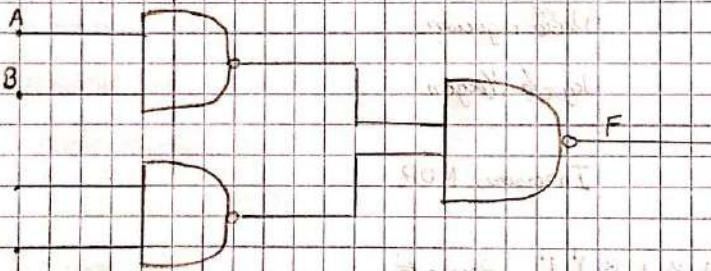
$$F = ((A \cdot B + C \cdot D)')'$$

Doble negación

$$((A \cdot B)' \cdot (C \cdot D)')'$$

Ley de Morgan

Circuito implementado con NAND



Ejercicio 2:

$$F = A + B$$

Desarrollo

$$F = ((A + B)')'$$

Doble negación

$$(A' \cdot B')$$

Ley de Morgan

$$A' = (A \cdot A)'$$

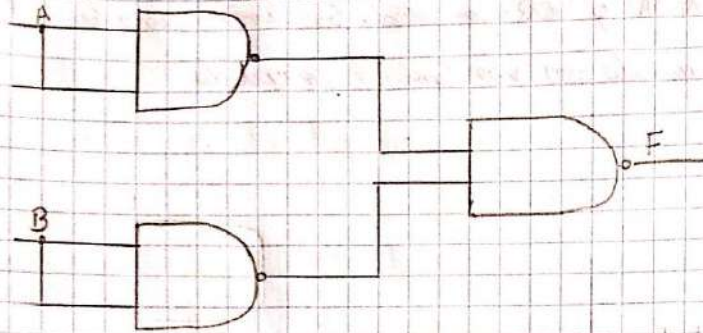
$$B' = (B \cdot B)'$$

Inversores NAND

$$F = ((A \cdot A)' \cdot (B \cdot B)')'$$

Repuesta

Circuitos implementados con NAND



3- Ejercicio 3

$$F = A \cdot B$$

Desarrollo

$$F = ((A \cdot B)')'$$

$$(A' + B')$$

Doble negación

Ley de Morgan

$$A' = (A + A)'$$

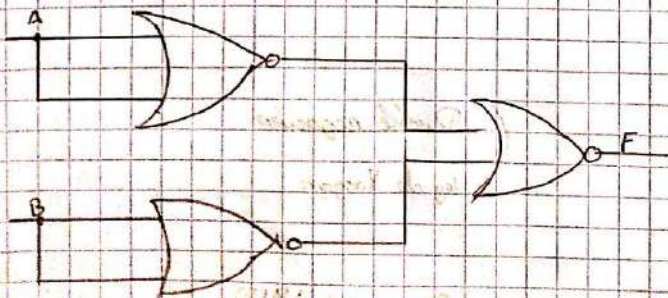
Inversores NOR

$$B' = (B + B)'$$

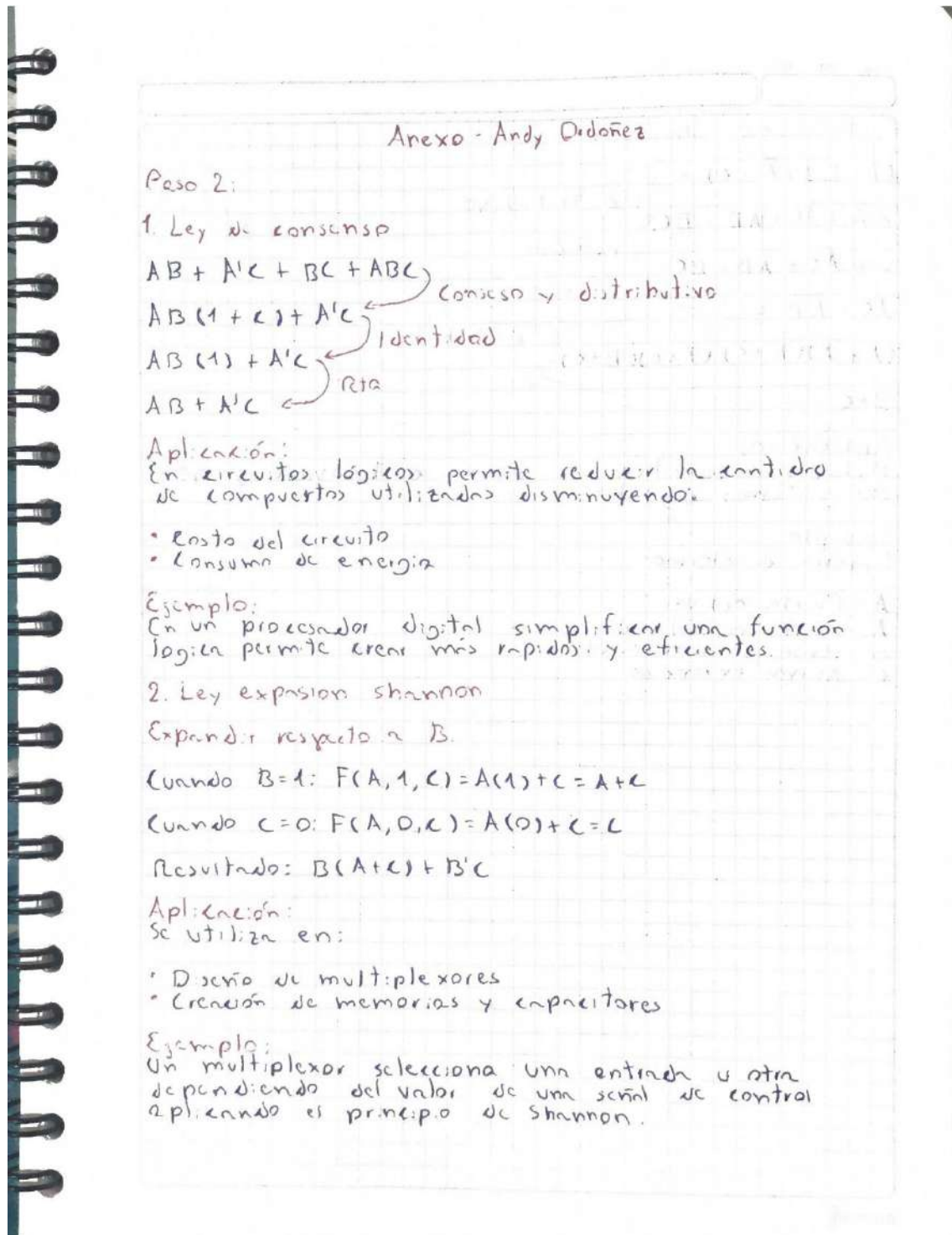
$$F = ((A + A)' + (B + B)')'$$

Respuesta

Circuitos implementados con NOR



✓ Andy Ordoñez



### 3. Resolución booleana

$$(A + B)(\bar{A} + C)$$

$$A\bar{A} + AC + \bar{A}B + BC$$

$$0 + AC + \bar{A}B + BC$$

$$AC + \bar{A}B$$

$$(A + \bar{A})(\bar{A} + C)(\bar{A} + C)(B + C)$$

$$B + C$$

distributiva

negación

consenso

distributiva

ata

Aplicación:

Metodos automaticos de diseño de hardware y optimizar sistemas digitales.

Ejemplo:

Sistema de alarma

A = Puerta activa

$\bar{A}$  = Puerta no activada

B = Sensor de movimiento

C = Alarma encendido.

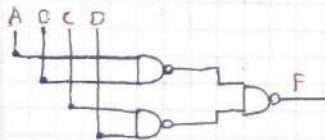
Paso 3:

Ejercicio 1

$$F = AB + CD$$

$$F = (\overline{\overline{AB + CD}}) \rightarrow \text{Doble negación}$$

$$F = (\overline{AB})(\overline{CD}) \rightarrow \text{Morgan}$$



Ejercicio 2:

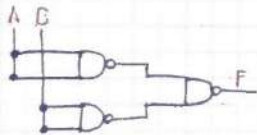
$$F = A + B$$

$$F = (\overline{\overline{A + B}}) \rightarrow \text{Doble negación}$$

$$F = (\overline{A \cdot B}) \rightarrow \text{Morgan}$$

$$\begin{aligned} \overline{A} &= \overline{A \cdot A} \\ \overline{B} &= \overline{B \cdot B} \end{aligned} \rightarrow \text{Inversores NAND}$$

$$F = (\overline{(\overline{A \cdot A})(\overline{B \cdot B})}) \rightarrow \text{Nota}$$



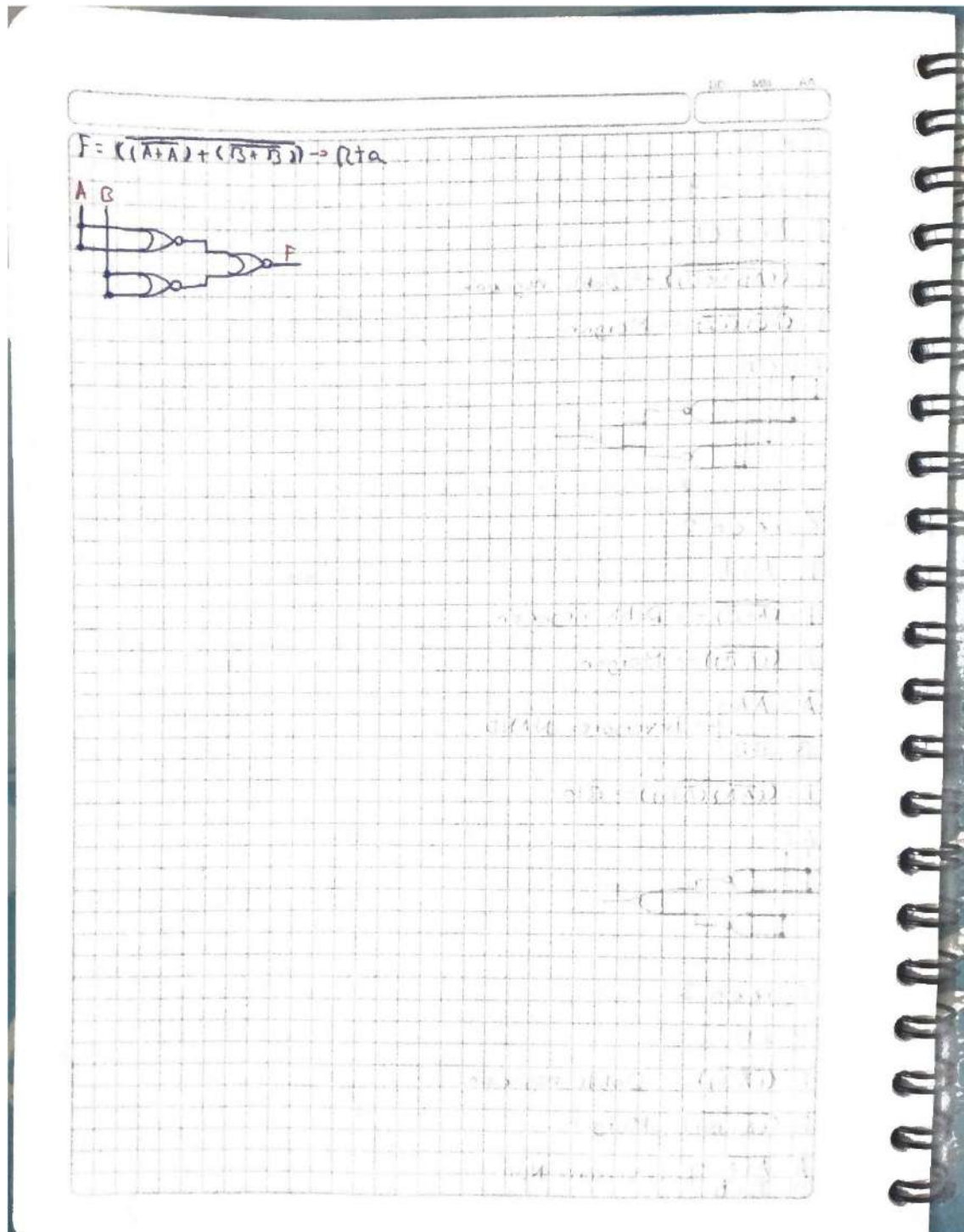
Ejercicio 3:

$$F = A \cdot B$$

$$F = (\overline{\overline{A \cdot B}}) \rightarrow \text{Doble negación}$$

$$F = (\overline{A + B}) \rightarrow \text{Morgan}$$

$$\begin{aligned} \overline{A} &= \overline{A + A} \\ \overline{B} &= \overline{B + B} \end{aligned} \rightarrow \text{Inversores NOR}$$



## Reporte Técnico de Actividades

### NRO 2 Fase

### 5 y 6.

1. los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital y programación

#### • Ley de Consenso

Aplicación: En circuitos lógicos permite reducir la cantidad de compuertas utilizadas disminuyendo, costos y consumo de energía.

Ejercicio:

$$AB + \bar{A}C + ABC + D$$

$$AB + \bar{A}C + ABC \rightarrow \text{Consenso}$$

$$AB(C + \bar{C}) + \bar{A}C \rightarrow \text{distributiva}$$

$$AB + \bar{A}C \rightarrow \text{Resultado}$$

#### • Ley Expansión Shannon

Aplicación: Se utiliza en diseño de multiplexores y en creación de memorias y capacitores

Ejemplo: Un multiplexor selecciona una entrada u otra dependiendo del valor de una señal de control aplicando el principio de Shannon.

Ejercicio:

$$F(A, B, C) = AB + C$$

Expandir respecto a "B"

1. Cuando  $B=1$ :  $F(A, 1, C) = AC + C = A + C$

2. Cuando  $C=0$ :  $F(A, B, 0) = AB + 0 = AB$

3. Resultado:  $B(A + C) + \bar{B}C$

• Resolución Booleana.

Aplicación: Métodos algorítmicos de diseño de hardware y obtención de sistemas de ecuaciones.

Ejemplo: Sistema de alarma.

A = Puerta activa

$\bar{A}$  = Puerta no activa

B = Sensor de movimiento

C = alarma encendida

$$(A + B)(\bar{A} + C) =$$

$$A\bar{A} + AC + \bar{A}B + BC \rightarrow \text{distributiva}$$

$$0 + AC + \bar{A}B + BC \rightarrow \text{negación}$$

$$AC + \bar{A}B \rightarrow \text{consenso}$$

$$(A + \bar{A})(\bar{A} + C)(\bar{A} + C)(B + C) \rightarrow \text{distributiva}$$

$$B + C \rightarrow \text{Rta.}$$

Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR: los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta.

### 1. Ejercicio 1

$$F = A.B + C.D$$

#### Desarrollo

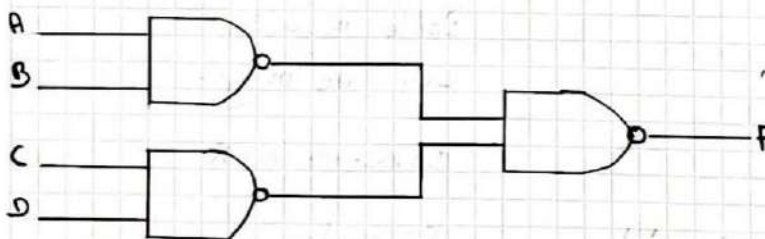
$$F = ((A.B)(C.D))'$$

$$((A.B)'.(C.D)')'$$

Doble negación

Ley de Morgan

#### Circuito implementado con NAND



### 2. Ejercicio 2

$$F = A + B$$

#### Desarrollo

$$F = ((A+B)')'$$

$$(A'.B')'$$

$$A' = (A.A)'$$

$$B' = (B.B)'$$

$$F = ((A.A)'.(B.B)')'$$

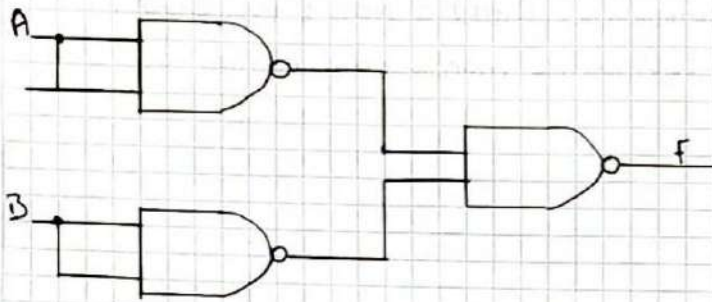
Doble negación

Ley de Morgan

Inversores NAND

Respuesta.

• Circuito Implementado con NAND



3 Ejercicio 3

$$F = A.B$$

• Desarrollo

$$F = ((A.B)')'$$

$$(A' + B')$$

$$A' = (A + A)'$$

$$B' = (B + B)'$$

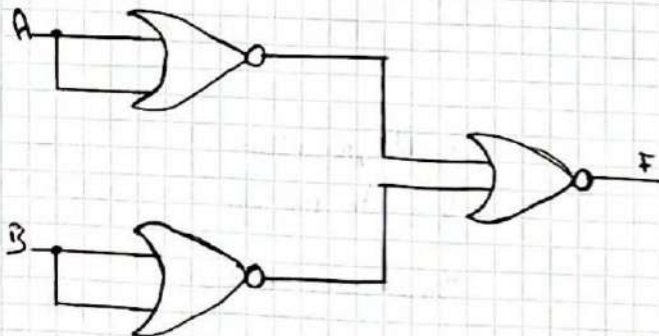
$$F = ((A + A)' + (B + B)')'$$

Doble negación  
Ley de Morgan

Inversores NOR

Respuesta.

• Circuito Implementado con NOR





Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

---